

UT 02.4

Estados y eventos en React con TypeScript

Dónde estamos en el temario

Unidad	Contenido	Estado
UT 02.1 – 02.2	Componentes, JSX, props, primer useState	Visto
UT 02.3	React con TypeScript: props, estado y eventos tipados	Visto
UT 02.4	Estados y eventos en profundidad, formularios básicos, elevar el estado	Hoy

Objetivos del tema

Entender

Por qué existe el estado y qué ocurre en cada renderizado

Decidir

Qué datos son estado y cuáles se calculan a partir de otros

Manejar

Eventos, formularios controlados y envío del formulario

Compartir

Estado entre componentes elevándolo al padre

Repaso exprés

Componente: función que devuelve JSX.

Props: datos que llegan desde el padre, de solo lectura.

Tipado: interfaz para las props y para el estado.

Opcional: `alCerrar?()`

```
interface PropsAlerta {  
  mensaje: string;  
  alCerrar?: () => void;  
}  
  
function Alerta({ mensaje, alCerrar }: PropsAlerta)  
{  
  return (  
    <div>{mensaje}</div>  
  );  
}
```

Sección 1

¿Por qué hace falta el estado?

Un componente por sí solo no recuerda nada entre renderizados.

Intento fallido: una variable normal

```
function Campo() {  
  let mensajeError = "";  
  
  const alSalir = () => {  
    mensajeError = "Obligatorio";  
  };  
  
  return (  
    <input onBlur={alSalir} />  
  );  
}
```

Qué pasa

La variable cambia, pero la pantalla no se actualiza.

Por qué

React no sabe que debe volver a ejecutar el componente.

La solución: useState

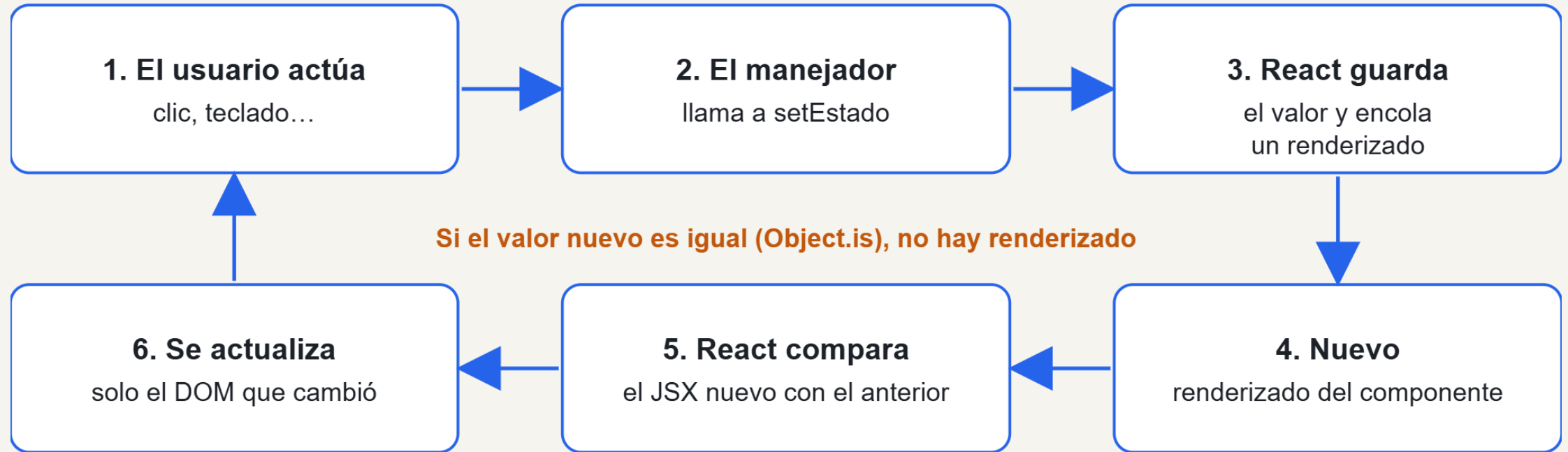
```
function Campo() {  
  const [mensajeError, setMensajeError] =  
    useState<string>("");  
  
  return (  
    <input  
      onBlur={() => setMensajeError("Obligatorio")}  
    />  
  );  
}
```

Idea clave

El estado es memoria del componente que, al cambiar, pide un nuevo renderizado.

La UI es una función del estado: **UI = f(estado)**.

El ciclo del estado



El estado es una instantánea

```
function Contador() {  
  const [cuenta, setCuenta] = useState(0);  
  
  const sumar = () => {  
    console.log("antes:", cuenta);  
    setCuenta(cuenta + 1);  
    console.log("después:", cuenta);  
  };  
  // ...  
}
```

Clic	antes	después
1.º	0	0
2.º	1	1
3.º	2	2

En cada renderizado, cuenta es una constante. El nuevo valor se verá en el **siguiente** renderizado.

StrictMode y componentes puros

En desarrollo

StrictMode renderiza cada componente dos veces para detectar efectos secundarios ocultos.

Componente puro

Mismas props y mismo estado producen el mismo JSX.
No se modifican variables externas durante el renderizado.

```
let veces = 0; // fuera del componente  
function Mal() { veces++; /* efecto en el render */ }
```

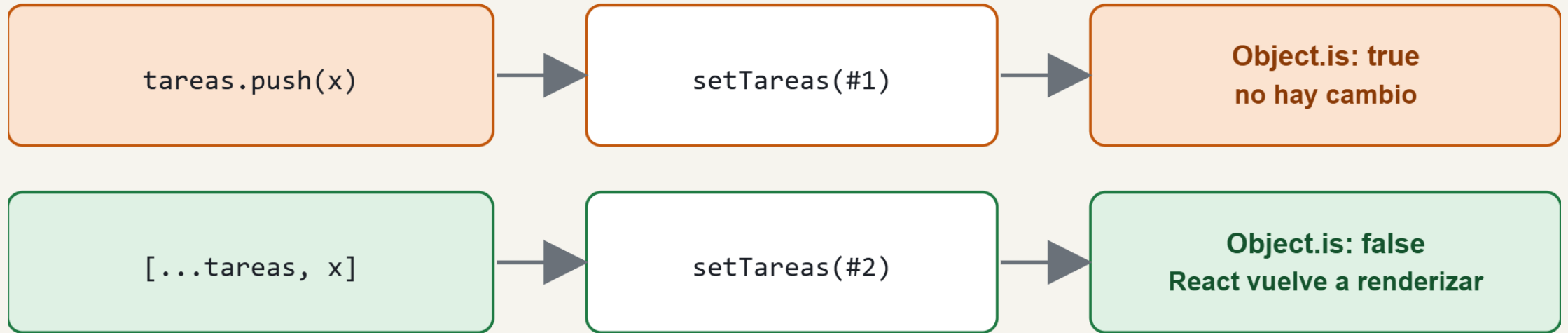
Regla 1: se reemplaza, no se fusiona

```
interface DatosAcceso {  
    usuario: string;  
    clave: string;  
}  
  
// Mal: se pierde "clave"  
setDatos({ usuario: "ana" });  
  
// Bien: copiamos y cambiamos  
setDatos({ ...datos, usuario: "ana" });
```

Diferencia con clases

En componentes de función, setX sustituye todo el valor anterior. Con objetos hay que copiar el resto de propiedades.

Regla 2: inmutabilidad



React compara referencias: hay que entregar un array u objeto **nuevo**.

Regla 3: forma funcional

Suma solo 1

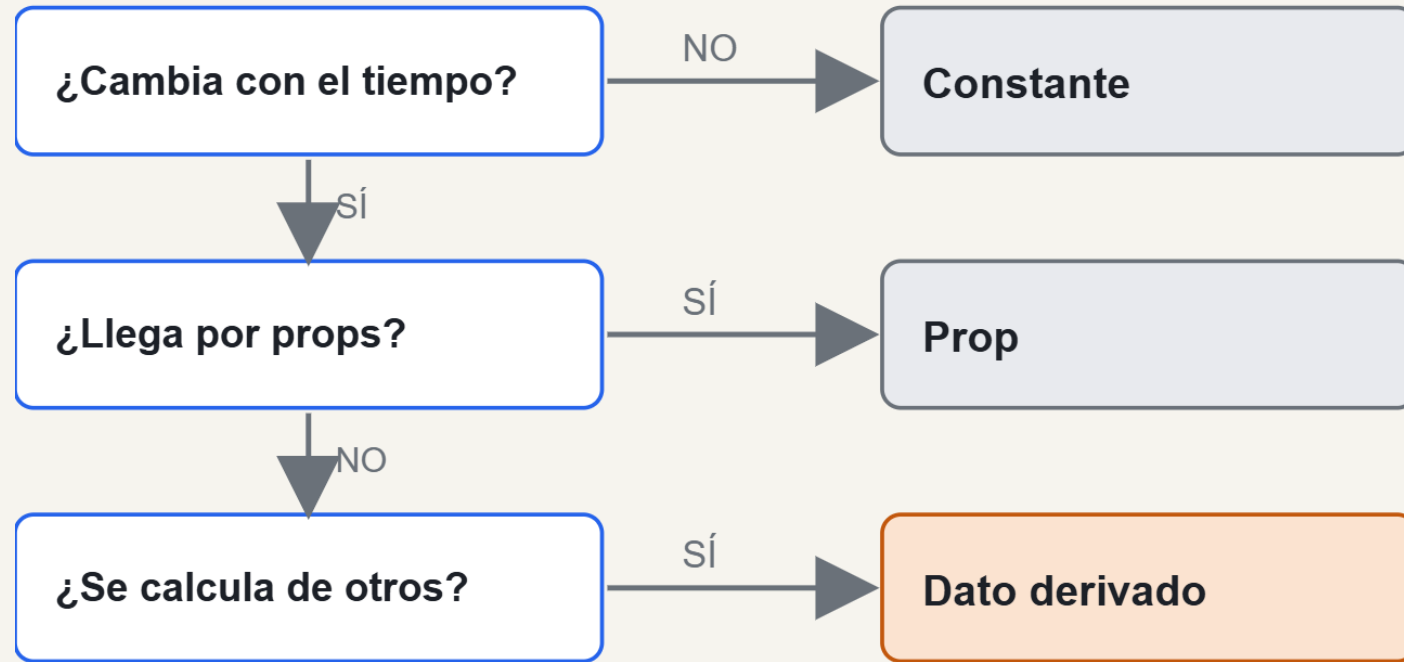
```
const sumarTres = () => {  
  setN(n + 1);  
  setN(n + 1);  
  setN(n + 1);  
};
```

Suma 3

```
const sumarTresBien = () => {  
  setN(prev => prev + 1);  
  setN(prev => prev + 1);  
  setN(prev => prev + 1);  
};
```

Las llamadas se agrupan (batching). Si el nuevo valor depende del anterior, pasa una función.

¿Es estado? Tres preguntas



Estado
si no es ninguno de los otros

Dato derivado: se calcula, no se guarda

```
function ListaProductos({ productos }: Props) {  
  const [filtro, setFiltro] = useState("");  
  
  // derivado: no es useState  
  const visibles = productos.filter((p) =>  
    p.nombre.includes(filtro)  
  );  
  // ...  
}
```

Un solo origen

Estado: filtro. Todo lo demás se recalcula.

Sin duplicar información no hay datos que se desincronicen.

Dos errores clásicos

Duplicar información

```
const [texto, setTexto] = useState("");  
const [largo, setLargo] = useState(0);  
  
// largo se puede calcular:  
const largo = texto.length;
```

Copiar una prop en estado

```
function Saludo({ nombre }: Props) {  
  const [n] = useState(nombre);  
  // si nombre cambia,  
  // n sigue con el viejo  
  return <p>{nombre}</p>;  
}
```

Sección 7

Eventos en profundidad

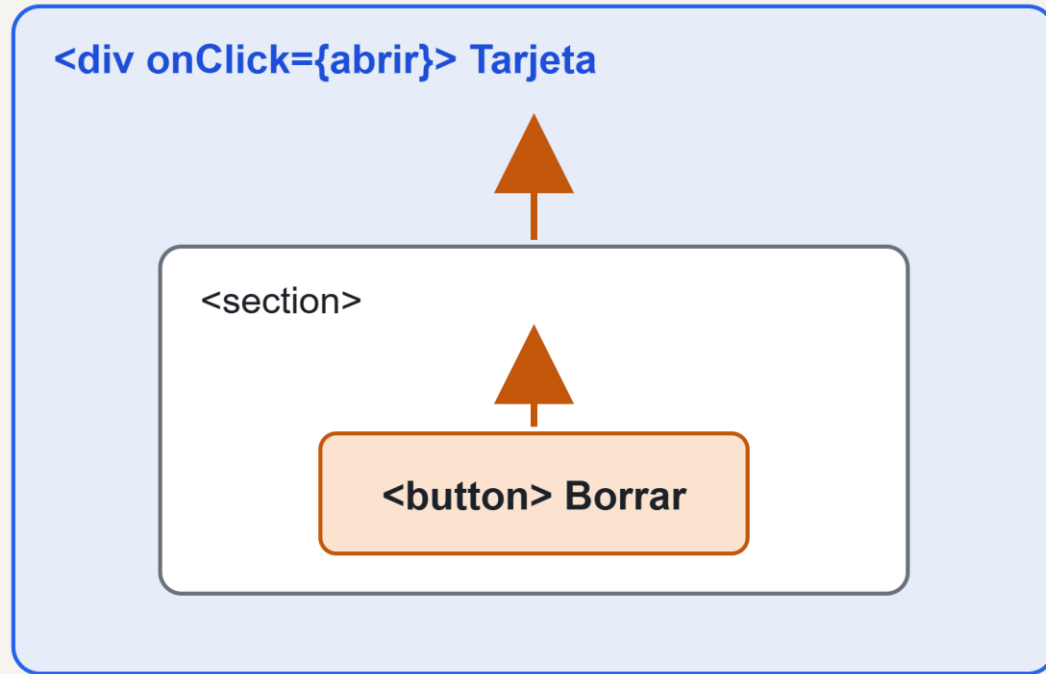
El objeto evento, el burbujeo y cómo pasar datos al manejador.

El objeto evento y los eventos habituales

Propiedad	Qué es
target	Elemento donde se originó
currentTarget	Elemento con el manejador
preventDefault()	Cancela la acción por defecto
stopPropagation()	Detiene el burbujeo

Evento	Cuándo se dispara
onClick	Clic
onChange	En cada pulsación
onSubmit	Envío del formulario
onKeyDown	Tecla pulsada
onBlur / onFocus	Pierde / gana el foco

Burbujeo de eventos



El clic del botón **sube** por sus ancestros y activa sus manejadores.

```
const borrar = (e: React.MouseEvent) => {  
  e.stopPropagation();  
  // ...  
};
```

preventDefault y pasar datos

Evitar recargar la página

```
const enviar = (e: React.FormEvent) => {  
  e.preventDefault();  
  // lógica propia  
};
```

Pasar un dato al manejador

```
// Bien: función flecha  
<button onClick={() => quitar(a.id)}>  
  
// Mal: se ejecuta al renderizar  
<button onClick={quitar(a.id)}>
```

Eventos de teclado

```
function BuscadorRapido({ alBuscar }: Props) {  
  const [texto, setTexto] = useState("");  
  
  const alPulsar = (  
    e: React.KeyboardEvent<HTMLInputElement>  
  ) => {  
    if (e.key === "Enter") alBuscar(texto);  
    if (e.key === "Escape") setTexto("");  
  };  
  // ...  
}
```

Propiedad útil

e.key devuelve el nombre de la tecla: "Enter", "Escape", "a"...

Campo controlado

```
const [correo, setCorreo] = useState("");

<label htmlFor="correo">Correo</label>
<input
  id="correo"
  type="email"
  value={correo}
  onChange={ (e) => setCorreo(e.target.value) }
/>
```

Fuente de verdad

El valor del input vive en el estado, no en el DOM.

Campo	Lee
text, email	value
checkbox	checked
select	value

Envío y validación derivada

```
function AltaBoletin() {  
  const [correo, setCorreo] = useState("");  
  const [enviado, setEnviado] = useState(false);  
  
  const errorCorreo = correo.includes("@")  
    ? "" : "Correo no válido";  
  
  const alEnviar = (e: React.FormEvent) => {  
    e.preventDefault();  
    if (!errorCorreo) setEnviado(true);  
  };  
}
```

Validación derivada

errorCorreo se calcula; no es estado.

Formularios complejos y librerías, más adelante.

El problema: dos componentes, un dato

BarraBusqueda

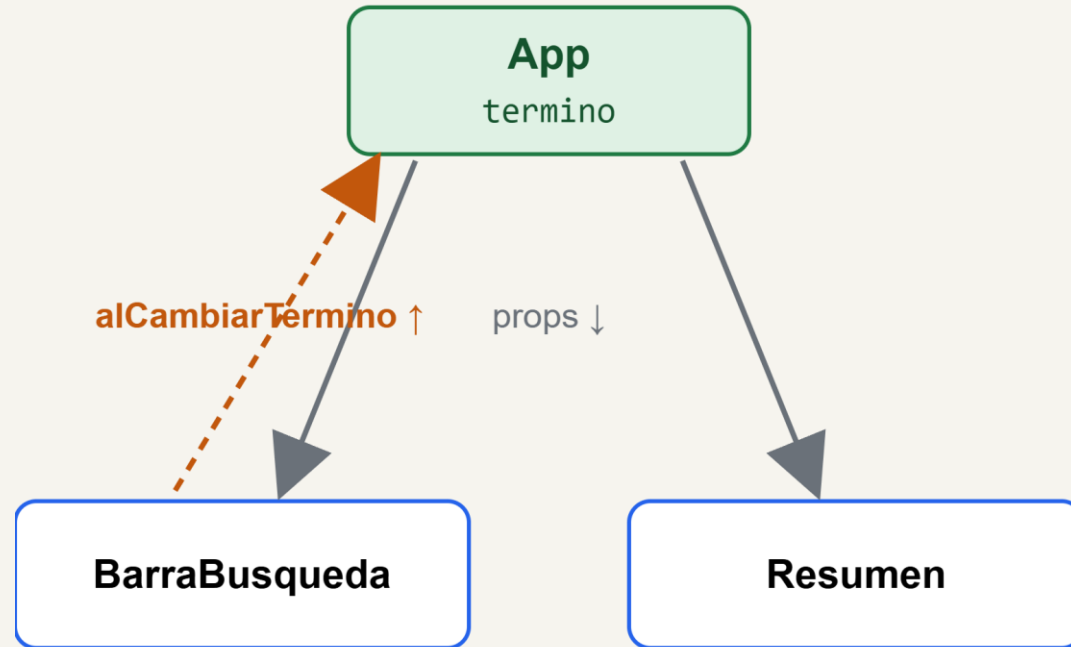
Tiene el input y guarda termino en su estado.

Resumen

Necesita mostrar el mismo termino, pero no puede leerlo.

Los hermanos no comparten estado. Los datos solo fluyen de padre a hijo.

La solución: elevar el estado



1. Buscar el ancestro común.
2. Mover allí el estado.
3. Bajar el valor por props.
4. Subir los cambios con un callback.
5. Los hijos pasan a ser controlados.

Errores frecuentes

Error	Corrección
Mutar el array o el objeto del estado	Crear una copia con spread, map o filter
Leer el estado justo tras setX	Usar el valor calculado o el siguiente render
onClick={funcion()}	onClick={funcion} o una función flecha
Copiar props en estado	Usar la prop directamente
Guardar datos derivados	Calcularlos en el renderizado
value sin onChange	Añadir onChange o usar defaultValue

Buenas prácticas

Estado mínimo

Guarda solo lo imprescindible y calcula el resto.

Nombres claros

Pares [valor, setValor] y manejadores manejarXxx.

Tipos explícitos

Tipa el estado cuando TypeScript no pueda inferirlo.

Estado cerca

Colócalo en el componente más bajo que lo necesite.

Preguntas de repaso

1. ¿Por qué una variable normal no sirve como estado?
2. ¿Qué diferencia hay entre target y currentTarget?
3. ¿Por qué push no actualiza la pantalla?
4. ¿Cuándo usar la forma funcional de setX?
5. ¿Cómo comparten estado dos componentes hermanos?

Resumen

- El estado es la memoria del componente; cambiarlo provoca un nuevo renderizado.
- Cada renderizado ve una instantánea; actualiza siempre sin mutar.
- Si se puede calcular, no es estado.
- Los eventos reciben un objeto tipado; controla el burbujeo y el comportamiento por defecto.
- Formularios controlados y estado elevado al padre común.

Anexo

Refuerzo del tipado de eventos

Cómo tipar con TypeScript los manejadores de eventos.

Inferir o anotar

En línea: TypeScript infiere

```
<input
  onChange={ (e) =>
    setTexto(e.target.value) }
/>
```

Función aparte: hay que anotar

```
function manejar(
  e: React.ChangeEvent<HTMLInputElement>
) {
  setTexto(e.target.value);
}
```

Anatomía del tipo

React.**MouseEvent**<**HTMLElement**>

espacio de nombres

tipo de evento

elemento del manejador

Propiedad	Tipo
e.currentTarget	HTMLElement, el elemento tipado en el genérico
e.target	EventTarget, puede ser cualquier descendiente

Tabla de referencia

Evento	Tipo del evento	Tipo del manejador
onClick	MouseEvent<HTMLButtonElement>	MouseEventHandler
onChange	ChangeEvent<HTMLInputElement>	ChangeEventHandler
onSubmit	FormEvent<HTMLFormElement>	FormEventHandler
onKeyDown	KeyboardEvent<HTMLInputElement>	KeyboardEventHandler
onBlur / onFocus	FocusEvent<HTMLInputElement>	FocusEventHandler
onMouseEnter	MouseEvent<HTMLDivElement>	MouseEventHandler

Eventos en las props

Estilo A: pasar el evento

```
interface Props {  
  alPulsar: React.MouseEventHandler<  
    HTMLButtonElement>;  
}
```

Estilo B (recomendado): pasar el dato

```
interface Props {  
  alCambiar: (nuevoValor: string) => void;  
}  
  
// el hijo llama:  
alCambiar(e.target.value);
```

El estilo B desacopla al padre del DOM. Para heredar todas las props de un botón: `React.ComponentProps<"button">`.

Errores de tipado habituales

Mensaje de TypeScript	Causa y solución
Parameter 'e' implicitly has an 'any' type	Función aparte sin anotar: tipar el parámetro
Property 'value' does not exist on type 'EventTarget'	Usar ChangeEvent o currentTarget
Type 'ChangeEventHandler' is not assignable...	Tipo del manejador distinto al evento
MouseEvent del DOM frente al de React	Usar React.MouseEvent, no el global
Property 'checked' does not exist (select)	checked solo existe en input checkbox

Ejercicios de tipado

1. Anotar

Pon el tipo correcto a cuatro manejadores.

2. Depurar

Explica el error de `e.target.value` en un `MouseEvent`.

3. Compartir

Un solo manejador para varias fichas con union de elementos.

4. Número

Campo numérico con `valueAsNumber`.